Architecture Client-Serveur et Programmation Middleware TP2	
Thème	Serveur TCP concurrent et expérimentation UDP.
Public visé	2ème Ingénieur
Chargée de cours	Amen Ben Hadj Ali
Chargées de TP	Amen Bidani – Asma Nafti- Arafet Zouari
Année universitaire	2026-2027

Objectifs généraux

Faire évoluer le serveur séquentiel du TP1 afin de traiter les requêtes de plusieurs clients de manière concurrente, puis réaliser une consultation de la même bibliothèque par UDP.
Le travail conserve les données, les services et le contrat applicatif du TP1. Il porte sur l'organisation des traitements et sur les mécanismes de communication, et non sur de nouvelles fonctionnalités métier.

Objectifs spécifiques

À l'issue de ce TP, l'étudiant devra être capable de :

- Distinguer traitement séquentiel, traitement concurrent et exécution simultanée de plusieurs requêtes.
- Déléguer le traitement d'un client TCP à une tâche, puis utiliser un pool de quatre threads.
- Identifier les données partagées et conserver les ressources de chaque échange dans sa tâche.
- Mettre en œuvre une consultation UDP et interpréter une absence de réponse.
- Comparer TCP et UDP à partir du code, des observations et des garanties du transport.

Résultat final attendu

Une évolution du serveur du TP1 permettant le traitement concurrent des requêtes de plusieurs clients, ainsi qu'une expérimentation de consultation par UDP et une comparaison argumentée avec TCP.

NB : Repartir d'un TP1 fonctionnel. Les données restent en mémoire et en lecture seule. La réservation, la persistance, les interfaces graphiques, RPC et RMI sont hors du périmètre de cette série.

Rappels du cours

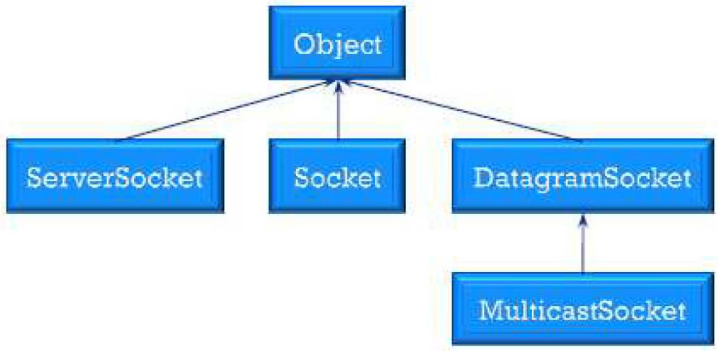
Exécution concurrente et partage des ressources

Dans le serveur séquentiel du TP1, le traitement d'un client se termine avant celui du suivant. Avec plusieurs threads, des traitements peuvent progresser pendant des périodes qui se chevauchent. La concurrence n'implique pas nécessairement une exécution parallèle sur plusieurs cœurs. [1]

Une tâche `Runnable` décrit un traitement par sa méthode `run()`. L'appel `start()` sur un objet `Thread` démarre un nouveau thread (fil d'exécution) ; appeler directement `run()` reste un appel ordinaire. `Runnable` est une interface technique d'exécution, pas une interface de service distante. [1]
Un pool réutilise des threads pour exécuter les tâches. La limite de quatre porte ici sur les tâches exécutées simultanément dans le pool, et non sur toutes les connexions TCP ouvertes. Les tâches supplémentaires peuvent attendre. [2]

Les sockets UDP en Java

La figure suivante regroupe les classes d'objet utilisées en Java pour la manipulation des sockets.



Le protocole UDP (se base sur les classes `DatagramSocket` et `DatagramPacket`) transporte des datagrammes dont les limites sont conservées. Il ne garantit ni la livraison, ni l'ordre, ni l'absence de doublons, et ne retransmet pas automatiquement les pertes. Un essai réussi sur le réseau local ne change pas ces garanties. [3]

Élément	Rôle dans l'expérimentation
`DatagramSocket`	Envoyer et recevoir des datagrammes ; aucun `accept()` n'est utilisé.
`DatagramPacket`	Associer des octets à une longueur et à une adresse / un port.
`setSoTimeout(...)`	Limiter l'attente d'un appel de réception côté client.

Ces éléments remplacent les échanges par flux de la version TCP. Le serveur UDP de ce TP restera séquentiel : le choix du transport est distinct du choix d'exécution. [4]

Présentation de l'étude de cas

La bibliothèque universitaire du TP1 reçoit désormais plusieurs consultations rapprochées. Conserver les cinq livres `L1` à `L5`, leurs titres et leurs thèmes, ainsi que les classes `Livre`, `Bibliotheque`, `CatalogueInitial` et `GestionBibliotheque`. Le client ne possède pas de catalogue local.

Opération conservée	Requête	Réponse
Consulter le catalogue	`LISTE`	`OK;5;identifiant:titre;...`
Rechercher par thème	`RECHERCHE;theme`	`OK;nombre;identifiant:titre;...` ou `OK;0`

Les commandes restent en majuscules. Les espaces supplémentaires des champs sont ignorés ; le thème entier est comparé sans distinction de casse. Conserver les erreurs `ERR;REQUETE_INVALIDE` et `ERR;COMMANDE_INCONNUE`, ainsi que les caractères réservés du TP1. Ne pas envoyer de menu ni de message d'accueil supplémentaire sur le réseau.

Exercice 1 — Du serveur séquentiel au serveur TCP concurrent

Objectif

Conserver le service et le protocole du TP1, tout en séparant l'acceptation des connexions du traitement de chaque client.

1. Observation de la version séquentielle

Reprendre le serveur final du TP1. Pour les essais uniquement, ajouter une attente simulée de 2 secondes après la lecture d'une requête et avant sa réponse. Afficher dans la console serveur le début et la fin du traitement, le nom du thread et l'adresse du client.
Lancer deux clients presque en même temps avec `RECHERCHE;JAVA`. Observer l'ordre des traitements. Conserver cette version comme référence, sans la lancer en même temps que la nouvelle sur le même port.

NB : L'attente simulée sert à rendre l'exécution observable ; elle ne fait pas partie du service métier. Elle sera désactivée pour les essais UDP. Adapter au besoin le délai de lecture des clients de test à 15 secondes pour inclure l'attente en file.

2. Création d'une tâche de traitement

Créer une classe `TraitementClientTCP` implémentant `Runnable` (ou bien par héritage de la classe `Thread`). Elle reçoit la socket acceptée et une référence au protocole partagé. Reprendre dans `run()` le traitement d'un client du TP1 : lecture, interprétation, réponse et fermeture de la connexion.

```
public class TraitementClientTCP implements Runnable / extends
Thread
public TraitementClientTCP(Socket socket,
                           ProtocoleBibliotheque protocole)
public void run()
```

Les signatures ci-dessus sont des repères à compléter, pas du code compilable. Chaque tâche conserve sa propre socket, ses flux et ses variables de requête/réponse. Une erreur d'échange est traitée dans cette tâche, sans fermer la socket d'écoute.

3. Adaptation du serveur

Créer `ServeurBibliothequeTCPConcurrent` à partir du serveur du TP1. Initialiser une seule fois le catalogue, le service, le protocole et la `ServerSocket`. Après chaque `accept()`, créer une tâche et démarrer un thread pour l'exécuter, puis revenir à l'acceptation du client suivant.
Ne pas appeler directement `run()` et ne pas attendre la fin du thread dans la boucle d'acceptation. Une fois la tâche démarrée, la fermeture de sa socket lui appartient : la boucle principale ne doit pas la fermer immédiatement.

4. Validation

Réutiliser le client TCP du TP1 sans changer son protocole : une ligne UTF-8 terminée par `\n`, une requête et une réponse par connexion. Avec deux clients et le délai simulé, vérifier que deux débuts de traitement peuvent apparaître avant la première fin. Les réponses métier restent identiques.
Questions: quels objets sont partagés ? Quelles variables doivent rester propres à chaque échange ? Pourquoi un thread par client ne constitue-t-il pas une politique de limitation de charge ?

Exercice 2 — Gestion des traitements par un pool de threads

Objectif

Réutiliser la tâche précédente et limiter à quatre les traitements TCP actifs dans un pool, sans modifier le client ni le contrat de consultation.

1. Mise en place du pool

Créer `ServeurBibliothequeTCPPool`. Remplacer la création d'un thread par connexion par un unique `ExecutorService`, initialisé avant la boucle d'acceptation :

```
ExecutorService pool = Executors.newFixedThreadPool(4);
```

Après chaque connexion acceptée, soumettre un nouvel objet `TraitementClientTCP` avec `execute(...)`, puis poursuivre l'écoute. Ne pas créer de pool dans la boucle et ne pas attendre le résultat de chaque tâche avant d'accepter le client suivant.

NB : `newFixedThreadPool(4)` utilise une file de tâches non bornée. Au plus quatre tâches du pool sont exécutées à la fois ; une cinquième peut attendre avec sa connexion déjà ouverte. Ce montage ne constitue pas une gestion complète de surcharge. [2]

2. Données partagées et ressources

Réutiliser une seule instance du catalogue et du service, initialisée avant la soumission des tâches. Aucune opération ne doit modifier les livres, leurs thèmes ou la collection pendant les essais. La liste de résultats et le formatage de la réponse sont propres à chaque appel.
Ne pas ajouter un verrou global autour de toute la lecture réseau ou de tout le traitement client : cela risquerait de sérialiser les échanges. Une future opération de réservation demanderait une analyse spécifique des accès concurrents ; elle n'est pas à développer ici.
Conserver une socket et des flux distincts par client. Si la soumission échoue, fermer la connexion qui n'a pas été confiée au pool. Prévoir `shutdown()` lorsque la boucle d'écoute se termine ; cet appel ne ferme pas lui-même les sockets. L'arrêt coordonné complet peut rester fourni par l'enseignant. [5]

3. Expérimentation

Conserver le délai simulé de 2 secondes dans la tâche. Lancer cinq clients avec une même requête, assez rapprochés pour que les quatre premiers traitements se chevauchent. Utiliser des consoles distinctes ou un lancement groupé organisé par l'enseignant.
Dans les journaux, relever les noms des travailleurs, les débuts et les fins. Vérifier que les cinq clients obtiennent une réponse et qu'un traitement supplémentaire attend lorsqu'il existe déjà quatre tâches actives. Ne pas confondre l'ordre des connexions, celui des traitements et celui des réponses.

Répéter un essai avec une requête invalide, puis un client valide. Le serveur doit rester opérationnel. Les temps observés servent à expliquer l'ordonnancement, pas à démontrer une accélération garantie.

4. Comparaison des versions TCP

Consigner brièvement le nombre de travailleurs (threads exécutants) utilisés, le chevauchement des traitements et l'attente observée. Distinguer le serveur séquentiel, le serveur avec un thread par connexion et le serveur à pool de quatre travailleurs.
Question: pourquoi peut-on avoir plus de quatre connexions ouvertes avec ce pool, alors que quatre tâches seulement sont exécutées simultanément ?

Exercice 3 — Consultation de la bibliothèque par UDP

Objectif

Conserver les opérations de consultation et leur format textuel, mais remplacer les flux TCP par des datagrammes. L'exercice est guidé ; il ne demande pas de reconstruire un transport fiable.

1. Convention d'échange

Créer `ServeurBibliothequeUDP` et `ClientBibliothequeUDP`. Chaque datagramme transporte une requête entière ou une réponse entière, encodée en UTF-8. **Ne pas ajouter `\n`** : la limite du datagramme délimite déjà le message.
Conserver `LISTE`, `RECHERCHE;theme` et les réponses du TP1. Par convention, limiter les messages à 1 024 octets encodés ; cette valeur est une limite pédagogique, pas la taille maximale du protocole UDP. Le petit catalogue fourni tient dans une réponse. Aucun découpage d'une réponse en plusieurs datagrammes n'est demandé.
Un client n'émet qu'une demande à la fois, sans réémission automatique. Il utilise une nouvelle socket pour chaque consultation, puis la ferme à l'issue de l'essai. Les échanges se déroulent sur le réseau pédagogique contrôlé.

2. Travail demandé côté serveur

Initialiser une fois le catalogue, le service et `ProtocoleBibliotheque`. Créer une `DatagramSocket` liée au port UDP `5001`, puis répéter les étapes suivantes :

- Préparer un paquet de réception avec un tableau de 1 025 octets et appeler `receive(...)`. Un paquet de cette capacité doit être créé ou réinitialisé avant chaque nouvelle réception.
- Si la longueur reçue dépasse 1 024 octets, répondre `ERR;REQUETE_INVALIDE`. Sinon, décoder uniquement la portion effectivement reçue et la transmettre au protocole.
- Encoder la réponse et l'adresser à l'IP et au port de l'émetteur obtenus dans le paquet reçu, puis continuer la boucle. Ne pas fermer la socket du serveur après chaque datagramme.

L'octet de marge permet de détecter un dépassement de la limite choisie, même si le datagramme est tronqué par le tampon. Réutiliser `getData()`, `getOffset()` et `getLength()` pour décoder les données utiles, et non la totalité du tableau. [4, 6]
Un datagramme de longueur nulle représente ici une requête vide et reçoit l'erreur de format prévue ; il ne signale pas une déconnexion. Le serveur UDP traite les datagrammes successivement et ne crée pas de thread par paquet.

3. Travail demandé côté client

Lire la requête, vérifier sa taille UTF-8, créer une `DatagramSocket`, puis envoyer un `DatagramPacket` vers l'adresse du serveur et le port `5001`. Fixer un délai de réception de 2 secondes avec `setSoTimeout(2000)`. [4]
Recevoir au plus une réponse avec un tampon de 1 025 octets ; vérifier sa longueur et l'adresse/port de provenance avant de l'afficher. Une provenance inattendue doit être signalée, sans interpréter son contenu comme un résultat du service. Fermer la socket après la réponse ou l'erreur.

4. Interprétation des erreurs

À l'expiration du délai, afficher « Aucune réponse reçue dans le délai imparti », et non « Aucun livre trouvé ». Une autre erreur réseau éventuelle doit être signalée séparément. Un délai expiré ne permet pas de conclure que le serveur est arrêté, ni de localiser une éventuelle perte. [3, 4]
Conserver la distinction entre une réponse `OK;0`, une réponse `ERR;...` et une absence de réponse. En UDP, il n'y a pas de lecture de fin de flux avec `readLine()`.

5. Essais guidés

Essai	Manipulation et observation demandées
Fonctionnement normal	Envoyer `LISTE`, `RECHERCHE;JAVA`, puis `RECHERCHE;HISTOIRE`. Comparer le contenu obtenu à celui de TCP.
Formats invalides	Essayer `RECHERCHE;` et `SUPPRIMER;L1`. Vérifier les mêmes erreurs applicatives qu'au TP1.
Absence de réponse simulée	Après avoir traité une requête, faire omettre volontairement un seul envoi par le serveur, en le journalisant. Constater le délai côté client, puis rétablir le fonctionnement normal.
Tampon volontairement trop petit	Réduire temporairement le tampon client à 32 octets et demander `LISTE`. Comparer la réponse reçue au résultat complet, puis rétablir la capacité de 1 025 octets.

L'omission volontaire simule une réponse manquante au niveau applicatif ; elle ne prouve pas une perte naturelle sur le réseau. Le tampon trop petit illustre une troncature à la réception, pas une fragmentation à programmer. Un préfixe `OK` ne suffit pas à conclure que la réponse tronquée est complète. [4]

6. Tableau comparatif à compléter

Compléter ce tableau dans le compte rendu. Séparer ce que les essais montrent de ce que les transports garantissent. Appuyer les réponses sur les rappels et les références. [3, 7]

Critère	Version TCP	Version UDP
Établissement de l'échange		
Délimitation des messages		
Livraison, ordre et doublons		

Critère	Version TCP	Version UDP
Absence de réponse et erreurs		
Taille des messages et réception		
Réutilisation du service métier		

NB : Ne pas conclure « UDP est toujours plus rapide » ou « UDP est fiable car tous les essais ont réussi ». Ne pas attribuer au transport une différence due au serveur TCP concurrent et au serveur UDP séquentiel.

Déploiement sur des postes distincts

1. Exécution sur le réseau

Placer le serveur sur un poste et les clients sur un ou plusieurs autres postes. Utiliser l'adresse réelle du serveur, et non `127.0.0.1`. Employer le port `5000` pour les essais TCP et `5001` pour UDP ; vérifier avec l'enseignant les règles d'accès propres à chaque transport. Ne pas désactiver globalement les protections.
Tester séparément les versions. Chaque serveur possède sa propre copie du même catalogue en lecture seule ; aucune synchronisation de données entre serveurs TCP et UDP n'est demandée. Désactiver l'attente simulée avant les comparaisons fonctionnelles avec UDP.

2. Vérifications finales

Point à vérifier	Résultat attendu
Deux clients TCP avec attente simulée	Chevauchement observable des traitements concurrents.
Cinq clients avec le pool de quatre	Au plus quatre tâches du pool actives ; réponse aux cinq clients.
Erreur puis consultation valide	Le serveur reste opérationnel.
Consultation UDP normale	Même contenu métier que dans la version TCP.
Réponse UDP volontairement omise	Délai signalé ; aucun faux `OK;0`.
Exécution sur plusieurs postes	Résultats conformes ; essai distinct de la boucle locale.

Synthèse et travail à remettre

Remettre le projet source, une notice d'exécution et un compte rendu court : conserver les trois versions TCP identifiables, joindre les journaux de concurrence, les résultats UDP et le tableau comparatif. Le code complet n'est pas à recopier dans le compte rendu.
Q1. Architecture et concurrence. Quelles responsabilités restent inchangées depuis le TP1 ? Pourquoi un pool de quatre threads ne limite-t-il pas à quatre toutes les connexions ouvertes ?

Q2. Données et ressources. Quels objets sont partagés et lesquels appartiennent à un seul échange ? Que faudrait-il réexaminer en ajoutant une réservation de livre ?
Q3. Communication. Distinguer une absence de résultat métier, une requête invalide et une absence de réponse. Pourquoi les essais ne suffisent-ils pas à prouver une garantie de livraison UDP ?

Références techniques

[1] Oracle — Thread : création, démarrage et gestion des threads en Java

[2] Oracle — Executors : création des exécuteurs et des pools de threads

[3] IETF — RFC 8085 : recommandations pour l'utilisation du protocole UDP dans les applications

[4] Oracle — DatagramSocket : envoi, réception et gestion des délais d'attente des datagrammes UDP

[5] Oracle — ExecutorService : soumission des tâches, gestion de leur exécution et arrêt du pool

[6] Oracle — DatagramPacket : représentation des datagrammes, tampon de données, longueur et adressage

[7] IETF — RFC 9293 : spécification du protocole TCP et transport fiable d'un flux d'octets